Day 11 - CSS Architecture: BEM, Organizing Stylesheets, Resets

Day 11 — CSS Architecture: BEM, Organizing Stylesheets, Resets

Objectives

- Understand the real problem large stylesheets run into, and why a naming convention like BEM exists to solve it
- Learn and apply BEM (Block, Element, Modifier) naming
- Understand how to organize CSS into multiple files as a project grows
- Understand what a CSS reset/normalize does and why browsers need one at all

The problem: CSS has no built-in way to prevent name collisions

Everything you've styled so far in this course has lived in small, single-purpose files, so naming collisions haven't been a real problem yet. But imagine a real website with 50 different components — a navbar, several kinds of buttons, cards, forms — all sharing one growing stylesheet. Two different developers (or even the same developer, weeks apart) might both reasonably create a class called `.title`, `.header`, or `.item` for two *completely unrelated* pieces of the page — and because CSS selectors aren't scoped to any particular section of the page by default, whichever rule has higher specificity (Day 3) or comes later in the file simply wins everywhere that class is used, silently breaking the other, unrelated use of that same name. This is a genuinely common, real problem in growing projects — not a hypothetical.

BEM (Block, Element, Modifier) is a naming *convention* — not a special CSS feature, just a disciplined way of naming your classes — specifically designed to prevent exactly this problem, by making every class name describe both what component it belongs to AND what role it plays within that component, so unrelated components can never accidentally collide.

BEM naming, explained through one real example

```
<div class="card">
    <img class="card__image" src="photo.jpg" alt="...">
    <h3 class="card__title">Product Name</h3>
    <p class="card__description">A short description.</p>
    <button class="card__button card__button--primary">Buy Now</button>
</div>
```

```
.card { }
.card__image { }
.card__title { }
.card__description { }
.card__button { }
```

```
.card__button--primary { }
```

- **Block** — a standalone, reusable component: `card`. This is the outermost, top-level name.
- **Element** — a part *of* that block, written as `block__element` (two underscores): `card__title` means "the title, specifically as it appears inside a card" — not just any generic title anywhere on the page.
- **Modifier** — a variation of a block or element, written as `block__element--modifier` (two dashes): `card__button--primary` means "the button inside a card, specifically the primary-styled variant" (as opposed to, say, a `card__button--secondary`).

Because every single class name is prefixed with its block (`card__...`), a completely unrelated part of the page — say, a navbar with its own `.navbar__title` — can never collide with `.card__title`, even though both are conceptually "a title" in plain English. The naming itself carries the necessary context to keep them apart, with zero risk of accidental overlap, and no need to rely on deeply nested selectors (`div.card > h3`) just to disambiguate them, which would otherwise increase specificity in exactly the way Day 3 warned against.

A genuinely practical side benefit: reading `class="card__button--primary"` directly in your HTML immediately tells you, without needing to go check the CSS at all, that this element is a button, specifically belonging to a card component, specifically the primary-styled variant — the naming itself documents the relationship.

Organizing CSS into multiple files as a project grows

For a small site (like everything you've built in the last ten days), one `styles.css` file is perfectly fine. As a real project grows, it's common to split styles into multiple files, organized by responsibility, and then combine them — either by linking several `<link>` tags in order, or, more commonly in real projects, using a build tool (outside this course's scope) that merges them into one file automatically. A common, sensible split:

```
styles/
■■■ reset.css          -- the reset/normalize (see below), always loaded FIRST
■■■ variables.css        -- your :root custom properties (Day 9)
■■■ base.css               -- body, headings, links -- broad, foundational defaults
■■■ layout.css               -- header, footer, page-level grid/flex structure
■■■ components/
■   ■■■ card.css               -- one file per BEM "block," e.g. all .card__ rules together
■   ■■■ button.css
■   ■■■ navbar.css
```

The exact split matters less than the underlying principle: **group related rules together, and keep the loading order intentional** (reset and variables first, since later files depend on them; broad/general rules before narrow/specific component rules, so specificity and source-order both work in your favor rather than against you, per Day 3).

CSS resets and normalize.css — why browsers need one at all

Here's a fact that surprises many beginners: **every browser ships with its own small, default stylesheet**, applied automatically to every page, before any of your own CSS runs at all — this is why an unstyled `<h1>` already appears large and bold, and an unstyled `<ul>` already has bullet points and left padding, even though

you never wrote a single rule for either. The problem: **these built-in default styles are not perfectly identical across different browsers** — a `<button>`, for instance, might have subtly different default padding, border style, or font in Chrome versus Firefox versus Safari.

A **CSS reset** (or the gentler, more modern **normalize.css**) is a stylesheet, loaded before any of your own styles, whose entire job is to flatten out these small, inconsistent browser defaults into one predictable, consistent baseline — so that the rest of your CSS behaves the same way regardless of which browser a visitor happens to be using. A minimal, commonly-used modern reset:

```
* {
    margin: 0;
    padding: 0;
    box-sizing: border-box;
}
```

This single, small rule — which you've actually already been using in some form since Day 3! — removes every browser's inconsistent default margin/padding on elements like `<body>`, headings, and lists, and applies the `border-box` sizing model everywhere, giving you a clean, entirely predictable starting point to build your own, deliberate spacing on top of, rather than fighting against invisible, browser-specific defaults you never asked for. Full reset libraries (like the well-known "Eric Meyer's Reset" or "normalize.css") go considerably further, addressing many more small inconsistencies across form elements, tables, and typography — worth knowing they exist and being able to add one to a real project, even though the simple version above covers the most common, impactful cases you'll run into in this course.

Avoiding specificity wars

A **specificity war** happens when developers respond to an unwanted CSS override by adding an even more specific selector (or, worse, `!important`) on top, which the next unwanted override then has to beat with something even more specific still — an escalating, increasingly unmanageable mess, exactly as warned about on Day 3. BEM directly helps prevent this: because every class name is already precisely scoped to its own component, you rarely need more than a single class selector (`.card__button--primary { }`) to target exactly the right element — there's no need to nest selectors deeply or add IDs just to "win" against some other unrelated rule, because BEM's naming convention already prevented the collision from ever being possible in the first place.

Exercises

Open `starter.html` and `starter.css`, follow the `<!-- TODO -->`/`/* TODO */` markers, and check your work against `CHECKLIST.md`.